

```
#include <stdio.h>

#define MAX 20

void sparseRepresentation(int a[][MAX], int rows, int cols, int b[][3])
{
    int i, j, k = 1;

    b[0][0] = rows;
    b[0][1] = cols;
    b[0][2] = 0;

    for(i = 0; i < rows; i++)
    {
        for(j = 0; j < cols; j++)
        {
            if(a[i][j] != 0)
            {
                b[k][0] = i;
                b[k][1] = j;
                b[k][2] = a[i][j];
                k++;
            }
        }
    }
    b[0][2] = k - 1;
}

void display(int b[][3])
{
    int i;
    printf("Row\tColumn\tValue\n");
    for(i = 0; i <= b[0][2]; i++)
    {
        printf("%d\t%d\t%d\n", b[i][0], b[i][1], b[i][2]);
    }
}

void sparseAddition(int a[][3], int b[][3], int c[][3])
{
    int i = 1, j = 1, k = 1;

    c[0][0] = a[0][0];
    c[0][1] = a[0][1];

    while(i <= a[0][2] && j <= b[0][2])
    {
        if(a[i][0] == b[j][0] && a[i][1] == b[j][1])
        {
            c[k][0] = a[i][0];
            c[k][1] = a[i][1];
            c[k][2] = a[i][2] + b[j][2];
            k++;
            i++;
            j++;
        }
        else if(a[i][0] < b[j][0] ||
                (a[i][0] == b[j][0] && a[i][1] < b[j][1]))
        {
            c[k][0] = a[i][0];
            c[k][1] = a[i][1];
            c[k][2] = a[i][2];
            k++;
            i++;
        }
        else
        {
            c[k][0] = b[j][0];
            c[k][1] = b[j][1];
            c[k][2] = b[j][2];
            k++;
            j++;
        }
    }
}
```

```
        {
            c[k][0] = b[j][0];
            c[k][1] = b[j][1];
            c[k][2] = b[j][2];
            k++;
            j++;
        }
    }

    while(i <= a[0][2])
    {
        c[k][0] = a[i][0];
        c[k][1] = a[i][1];
        c[k][2] = a[i][2];
        k++;
        i++;
    }

    while(j <= b[0][2])
    {
        c[k][0] = b[j][0];
        c[k][1] = b[j][1];
        c[k][2] = b[j][2];
        k++;
        j++;
    }

    c[0][2] = k - 1;
}

void sparseTranspose(int a[][3], int b[][3])
{
    int i, col, k = 1;

    b[0][0] = a[0][1];
    b[0][1] = a[0][0];
    b[0][2] = a[0][2];

    for(col = 0; col < a[0][1]; col++)
    {
        for(i = 1; i <= a[0][2]; i++)
        {
            if(a[i][1] == col)
            {
                b[k][0] = a[i][1];
                b[k][1] = a[i][0];
                b[k][2] = a[i][2];
                k++;
            }
        }
    }
}

int main()
{
    int rows, cols;
    int mat1[MAX][MAX], mat2[MAX][MAX];
    int s1[401][3], s2[401][3], result[401][3], trans[401][3];
    int i, j;

    printf("Enter number of rows and columns: ");
    scanf("%d%d", &rows, &cols);

    printf("Enter First Matrix elements:\n");
    for(i = 0; i < rows; i++)
    {
```

```
for(j = 0; j < cols; j++)
{
scanf("%d", &mat1[i][j]);
}
}

printf("\nEnter Second Matrix elements:\n");
for(i = 0; i < rows; i++)
{
for(j = 0; j < cols; j++)
{
scanf("%d", &mat2[i][j]);
}
}

sparseRepresentation(mat1, rows, cols, s1);
printf("\nSparse Matrix Representation of First Matrix:\n");
display(s1);

sparseRepresentation(mat2, rows, cols, s2);
printf("\nSparse Matrix Representation of Second Matrix:\n");
display(s2);

sparseAddition(s1, s2, result);
printf("\nSparse Matrix After Addition:\n");
display(result);

sparseTranspose(s1, trans);
printf("\nTranspose of First Sparse Matrix:\n");
display(trans);

return 0;
}
```